SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	**Total Marks:**	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse realworld problem.*		
Student Name:	SRIVATSAN L	**Reg. No.:**	192521480
Section / Batch:	2nd	**Date:**	02/09/2026

Code of Conduct

I SRIVATSAN L (192521480) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: SRIVATSAN L

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

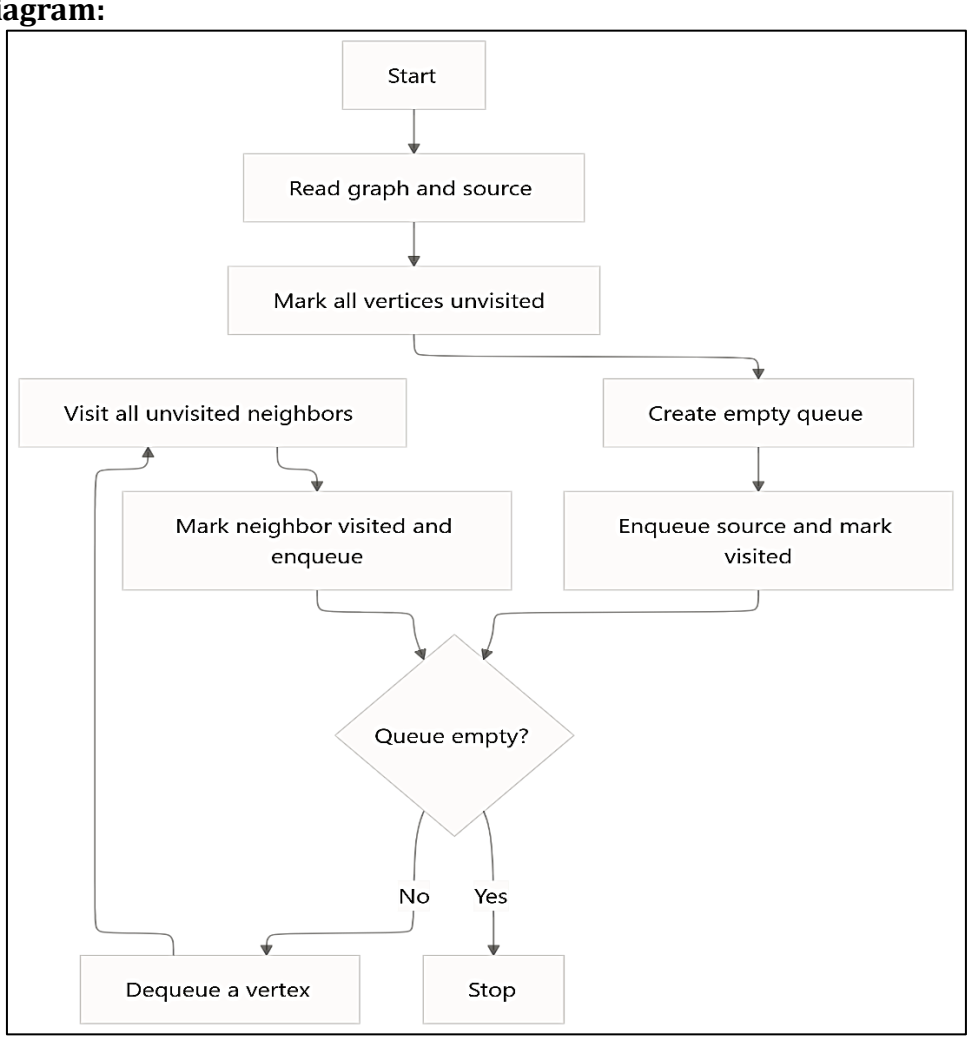
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20

GRAND TOTAL:	100 Marks

Annexure A – Flowchart-To-Code Conversion

1. Convert the flowchart for Breadth First Search traversal into code. ([1]0 Marks)

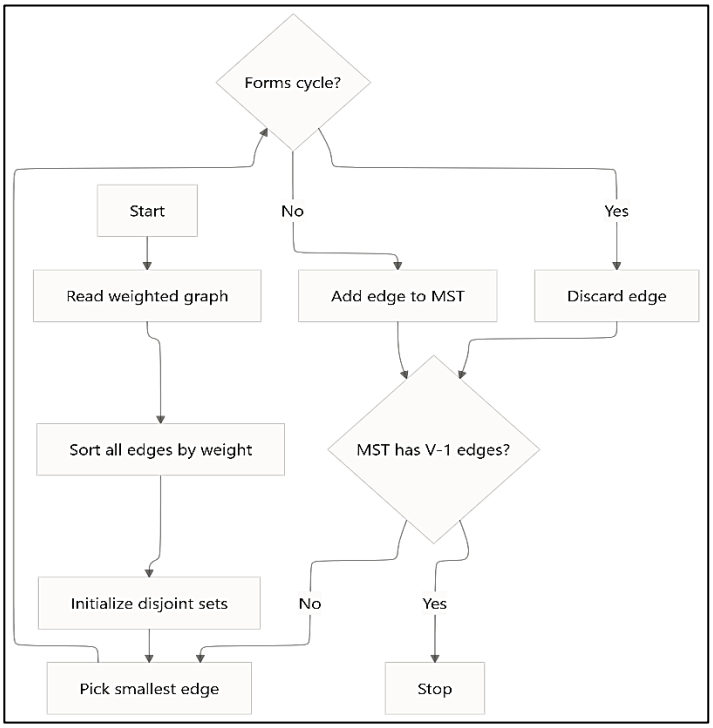
Flowchart Diagram:



---

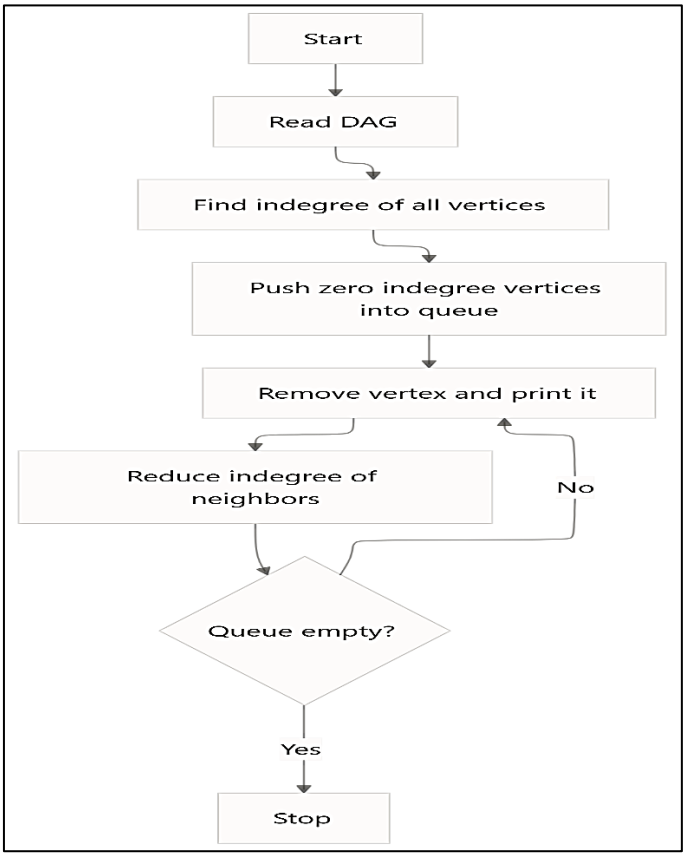
[1] . Convert the flowchart for Kruskal's algorithm into code. (20 Marks)

Flowchart Diagram:



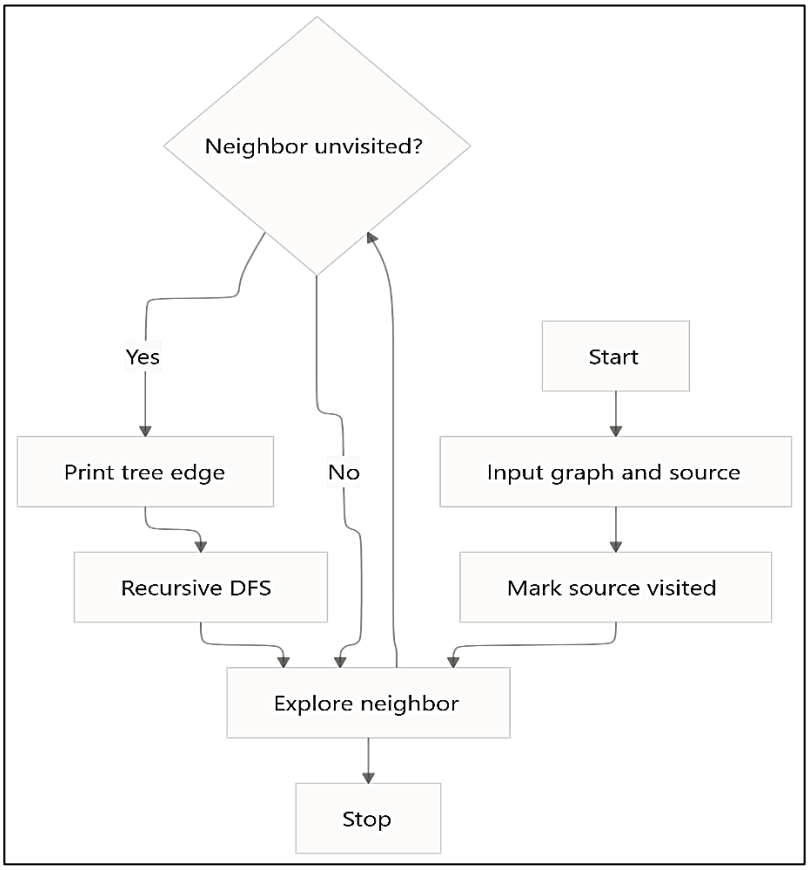
3. Convert the flowchart for topological sort into code. (20 Marks)

Flowchart Diagram:



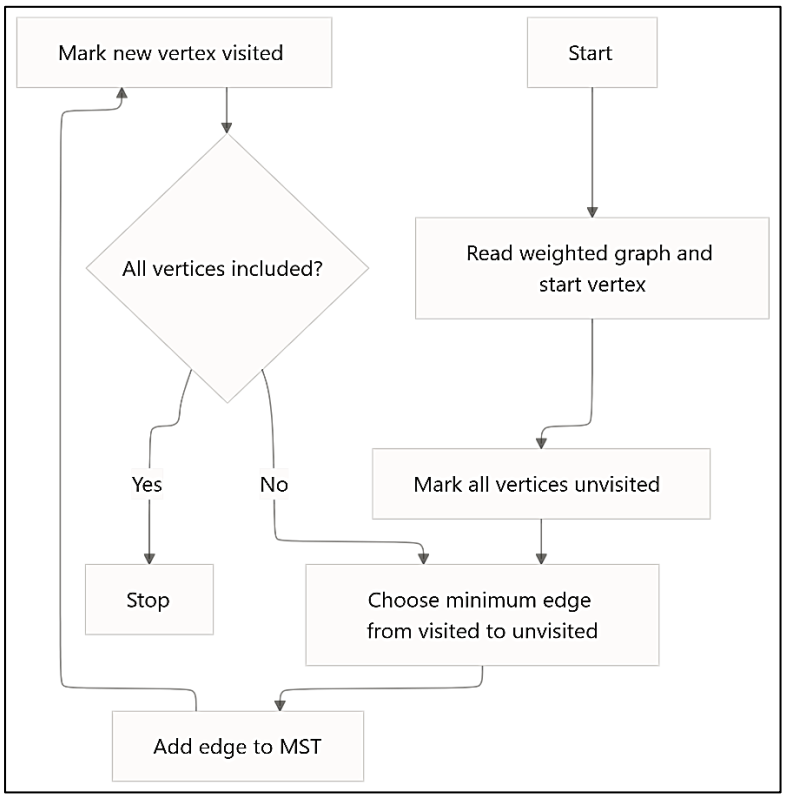
4. Convert the flowchart for generating a DFS tree into code. (20 Marks)

Flowchart Diagram:



5. Convert the flowchart for Prim's algorithm into code. (20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or nonfunctional code
Code Quality & Readability	20	Clean, modular, wellcommented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Breadth first search [BFS]

Aim: To perform Breadth first search traversal of a graph using a queue.

Algorithm:

- Read the graph and source vertex
- mark all vertices as unvisited
- Insert source into queue & mark it visited
- Delete a vertex & print
- go through all
- Repeat untill the queue become empty

C- program:

```
#include <stdio.h>
#define MAX 20
int main(){
    int a[MAX][MAX]={0}, v[MAX]={0}, q[MAX], n,e,
                    i,u,w,s, f=0, r=0, x;
    scanf("%d%d",&n,&e);
    for(i=0; i<e; i++){ scanf("%d%d",&u,&w);
     a[u][w]=a[w][u]=1; }
    scanf("%d",&s); q[r++]=s; v[s]=1;
    while(f<r){
        x=q[f++]; printf("%d",x);
        for(i=0; i<n; i++)
            if(a[x][i]&&!v[i])
                v[i]=1, q[r++]=i; }
    return 0; }
```

Sample Input and output

6 4 BFS: 0 1 2 3 4
0 1
0 2
0 3

2.) Kruskal's Algorithm

Aim: To find the minimum spanning tree of a weighted graph using Kruskal.

Algorithm:

- Read the weighted graph
- Sort all
- put vertex in seperate set
- Select at smallest edge.
- If not add - MST
- Repeat until all edges are selected.

C-program :

```
#include <stdio.h>
struct E{ int u,v,w} e [10], t;
int p[10], f(int x) { return p[x] == x ? x : p[x] = f(p[x]); }

int main () {
    int n, m, i, j, a, b, c = 0;
    scanf("%d %d", &n, &m);
    for (i=0; i<m; i++) scanf ("%d %d %d", &e[i].u, &e[i]
                                .v, &e[i].w);

    for (i=0; i<m; i++) for (j = i+1; j < m; j++)
        if (e[i].w > e[j].w) t = e[i], e[i]
                              = e[j], e[j] = t;

    for (i=0; i<n; i++) p[i] = i;

    for (i=0; i<m && c < n-1; i++)
        if (a = f(e[i].u)) != (b = f(e[i].v))
            p[a] = b, printf ("%d - %d \n",
                e[i].u, e[i].v), c++;

}
```

Sample I/P & O/P :

I/P

```
4 5
1 2 10
1 3 5
1 3 6
2 4 15
3 4 7
```

O/P

```
1 - 3
2 - 3
3 - 4
```